

P1688R0

Towards a C++ Ecosystem Technical Report

Published Proposal, 2019-06-16

Author:

[Bryce Adelstein Leibach](#) (NVIDIA)

Source:

[GitHub](#)

Issue Tracking:

[GitHub](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

EWG, SG15

Table of Contents

1 Introduction

2 Timeline

3 Outline

4 Format

§ 1. Introduction

At the ISO C++ Kona 2019 meeting, the ISO C++ Committee's Tooling Study Group, SG15, met to discuss concerns raised by various stakeholders about how modules would impact and interact with the broader C++ ecosystem (build systems, tools, other languages, etc). During that discussion, SG15

reached a consensus that the best way to prepare the C++ community for modules and ensure a smooth transition to modules over the next decade would be to prepare a C++ Ecosystem Technical Report on modules.

Since that meeting, the Tooling Study Group has held a series of telecons to plan, discuss, and brainstorm ideas for the proposed Technical Report. One of the outcomes of these telecons is an outline and high-level design for the proposed Technical Report, as described below.

§ 2. Timeline

The Technical Report should be feature-driven not deadline-driven; we should ship it when it's ready.

The Technical Report will be most useful if it is released **after** we have had enough field experience to write the Technical Report, but **before** the C++ community begins widespread adoption of modules, so that the report is available to aid during the transition.

§ 3. Outline

We suggest that the Technical Report be split into three sections:

- **Usage:** Explains the requirements and expected usage of modules across the C++ ecosystem.
Raises questions which need to be addressed later in the document.
- **Stakeholders:** Different types of users of modules.
 - C++ Implementations:
 - GCC, Clang/LLVM, Visual Studio, EDG, PGI, ICC, xIC.
 - Build Systems:
 - CMake, Make, Boost Build, autoconf, shell scripts, Meson, Ninja, Scons, Waf, Bazel, Buck, Cargo, Gulp, Webpack, Ant, Ibuild, Evoke, qmake, MSBuild, internal company build systems, mixed build systems, distributed build systems (icecc, ccache, sccache, incredibuild, distcc, fastbuild, Bazel remote build execution).
 - Tools:
 - IDEs, Clang-based tools, CastXML, static analysis tools (Coverity, Clang Static Analyzer, Grammatech, cppcheck), code generation tools (QT Moc, Protobuf), test

frameworks (Google Test), test case reduction tools (creduce, delta).

— Libraries:

— Boost, header only libraries.

— Distributions:

— vcpkg, conan.io, Linux distributions (RPM-based, Debian-based).

— Other Languages:

— CUDA, OpenCL/SYCL, C, Python, Rust, Java, SWIG.

— End Users.

— **Archetypes:** Concrete examples based on expected usage.

— Hello world with modules.

— Header only library.

— Incremental build.

— Distributed build.

— Building Compiled Module Interfaces (CMIs) only for tooling consumption.

— Dependency scanning vs explicit module dependencies build.

— **Findings:** Focused technical sections that explore open questions in detail and present the results of field experience. Unopinionated.

— **Module Mapping:** What approaches and formats are effective for communicating module name <-> module-interface/header file mappings? Module name + configuration <-> Compiled Module Interface (CMI) mappings?

— **Module Naming:** How should module names be structured? How do we avoid conflicts between different projects? How do we deal with versioning?

— **Module ABI:** How do we maintain stable ABIs in a modular world?

— **Codebase Transition Path:** How should projects transition from headers to modules? How should projects support both pre-C++20 headers and C++20 modules?

— **Compiled Module Interface (CMI) Configuration:** How do we find the CMI that was compiled in the same way as the current TU? What defines the configuration of a CMI?

— **Compiled Module Interface (CMI) Distribution:** How effective is the distribution of CMIs alongside module-interface/header files?

- **Dependency Scanning:** How do we do dependency scanning in a modular world? Can we make it fast?
- **Build Performance:** How do modules impact build performance? What impact does modules have on parallelism in C++ builds?
- **Module Granularity:** What size should modules be to maximize performance and usability? Does the cost of an import scale with the size of the module?
- **Guidance:** Concise set of guidelines for the C++ ecosystem. Addresses questions raised in Usage and draws conclusions based on results from Findings.

§ 4. Format

The TR will likely need to be in Latex, using a fresh fork of the IS Latex that has been customized, similar to the Coroutines TS Latex. We have had very painful issues with non-Latex formal documents in the past. For example, the Parallelism TS v2 was originally written in HTML which was converted to PDF for publication. It had to be completely rewritten in Latex after we voted to publish it because of typesetting issues raised by ISO that could not be resolved.